

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



BattleScript: Lenguaje Declarativo Minimalista para Batallas Estratégicas por Turnos

Ponderacion: 45pts

Horas aproximadas: xx

Indice

1. Objetivos	3
2. Descripción	3
3. Introducción	4
4. Estructura del lenguaje	6
5. Sistema de tipos	8
6. Sección de definición de estrategias	10
7. Sección de definición de partidas.....	20
8. Sección de punto de entrada	25
9. Entorno de Trabajo	28
10. Funcionamiento de la semilla aleatoria	29
11. Reportes	30
12. Requerimientos mínimos	30
13. Entregables.....	31
14. Restricciones	32
15. Fecha de entrega	32

1. Objetivos

1.1 Objetivo general

Aplicar los conocimientos relacionados con las fases de análisis léxico y sintáctico de un compilador mediante el desarrollo de un lenguaje declarativo capaz de definir y ejecutar estrategias automáticas en combates por turnos.

El sistema deberá analizar archivos con extensión **.bt** escritos en un lenguaje propio, validar su estructura y ejecutar las acciones gramaticales necesarias para simular las partidas definidas por el usuario.

1.2 Objetivos específicos

- Desarrollar un analizador léxico utilizando JFlex.
 - Desarrollar un analizador sintáctico utilizando CUP.
 - Aplicar los conceptos de token, lexema, patrón y expresión regular.
 - Diseñar una gramática libre de ambigüedades para representar estrategias, partidas y ejecuciones.
 - Implementar reglas condicionales mediante expresiones booleanas.
 - Manejar correctamente la precedencia de operadores.
 - Ejecutar acciones gramaticales escritas en Java.
 - Implementar funciones para consultar el historial de movimientos.
 - Simular combates automáticos entre dos estrategias.
 - Utilizar un generador pseudoaleatorio determinista para reproducir exactamente los resultados.
 - Generar reportes de tokens y errores léxicos.
 - Construir una interfaz gráfica que permita crear, abrir, guardar y ejecutar archivos.
-

2. Descripción

El estudiante deberá desarrollar una aplicación que permita definir y ejecutar estrategias automáticas para un juego de batalla por turnos.

En este juego existirán dos clases de combatientes:

- Magos.
- Guerreros.

Cada estrategia estará formada por:

- Un nombre.
- Una clase.
- Una acción inicial.
- Una colección ordenada de reglas.
- Una acción por defecto.

Las reglas permitirán analizar el estado actual de la batalla, la vida disponible, los recursos, el historial de acciones y un valor pseudoaleatorio. A partir de estas

condiciones, cada estrategia seleccionará automáticamente el movimiento que desea realizar.

El lenguaje permitirá definir:

1. Estrategias de magos y guerreros.
2. Partidas entre dos estrategias.
3. Sistemas de puntuación.
4. Bonificaciones por secuencias de movimientos.
5. Un punto de entrada para ejecutar las partidas.
6. Una semilla que permita repetir exactamente los resultados.

La aplicación deberá procesar los archivos utilizando JFlex y CUP. Después de completar correctamente el análisis léxico y sintáctico, deberá ejecutar las partidas en el orden especificado dentro del bloque main.

No se requiere una fase independiente de análisis semántico ni recuperación de errores.

Las restricciones que puedan expresarse mediante la gramática deberán validarse sintácticamente.

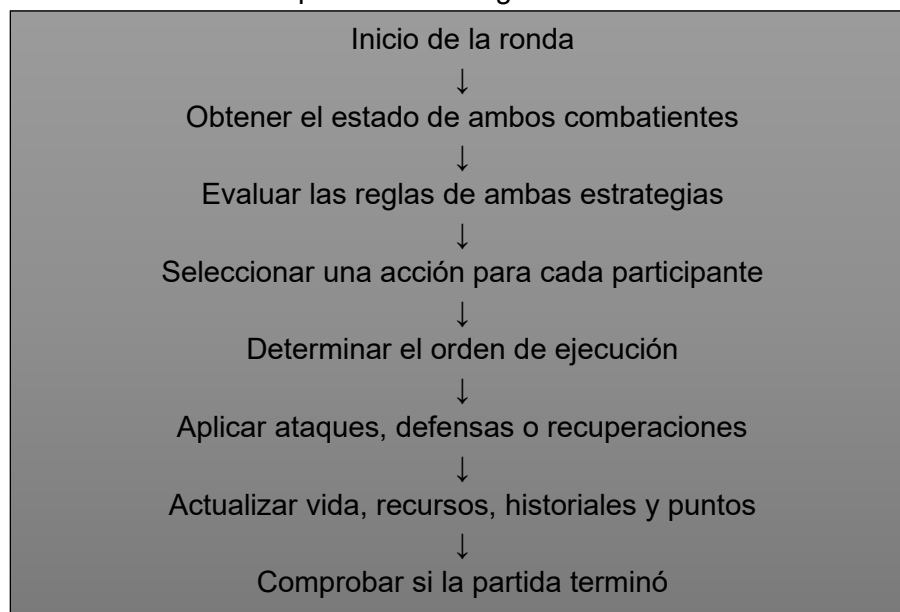
3. Introducción

3.1 ¿Qué es una batalla estratégica por turnos?

Una batalla por turnos es un sistema en el cual dos combatientes seleccionan una acción durante cada ronda. Después de que ambos han elegido, el sistema determina el orden de ejecución y aplica los efectos correspondientes.

A diferencia de un juego de acción en tiempo real, los participantes no actúan continuamente. El combate se divide en rondas claramente definidas:

En BattleScript, los jugadores no seleccionarán manualmente sus movimientos. Cada combatiente será controlado por una estrategia declarativa.



Por ejemplo:

Si mi vida es menor o igual a 30 y tengo suficiente maná,
utilizar una runa de curación.

Si el rival está cerca de ser derrotado,
utilizar una bola de fuego.

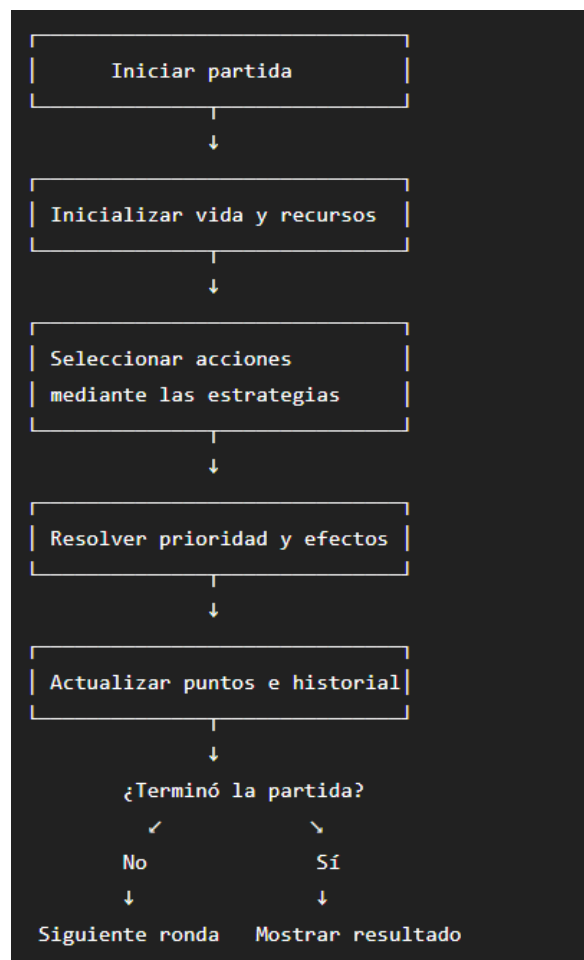
Si tengo poco maná,
meditar.

En cualquier otro caso,
utilizar un proyectil arcano.

Esta lógica se expresará mediante reglas if, then y else.

Figura sugerida 1: flujo general del combate

Puedes colocar este contenido como diagrama:



4. Estructura del lenguaje

El lenguaje estará compuesto por tres secciones principales:

1. Definición de estrategias.
2. Definición de partidas.
3. Punto de entrada.

Ejemplo visual de la estructura

1. ESTRATEGIAS

```
mage Merlin { ... }  
warrior Ragnar { ... }
```



2. PARTIDAS

```
match DueloFinal { ... }
```



3. PUNTO DE ENTRADA

```
main {  
  run [DueloFinal] with { ... }  
}
```

Ejemplo completo de entrada

```
mage Merlin {
  initial: ARCANE_BOLT

  rules: [
    if self_resource <= 20 then MEDITATE,
    else ARCANE_BOLT
  ]
}

warrior Ragnar {
  initial: SLASH

  rules: [
    if self_resource <= 20 then REST,
    else SLASH
  ]
}

match DueloInicial {
  players: [Merlin, Ragnar]
  rounds: 5

  scoring: {
    damage_point: 1,
    healing_point: 1,
    successful_defense: 20,
    victory_bonus: 100,
    failed_action_penalty: 10
  }

  bonuses: {
    mage_combo: [ARCANE_BOLT, ARCANE_BOLT, FIREBALL],
    mage_combo_points: 35,
    warrior_combo: [SLASH, SLASH, HEAVY_STRIKE],
    warrior_combo_points: 40,
    low_health_victory: 25
  }
}

main {
  run [DueloInicial] with {
    seed: 42
  }
}
```

4.1 Comentarios

Los comentarios serán ignorados por el analizador léxico y no se enviarán al analizador sintáctico.

4.1.1 Comentarios de una línea

Comienzan con `//` y finalizan al encontrar un salto de línea.

```
// Estrategia de un mago defensivo
mage Merlin {
  initial: MAGIC_BARRIER
}
```

4.1.2 Comentarios multilínea

Comienzan con `/*` y terminan con `*/`. Si un comentario multilínea no se cierra, deberá registrarse un error léxico y detenerse el análisis.

```
/*
  Esta estrategia utiliza magia defensiva
  cuando el guerrero realiza ataques fuertes.
*/
mage Merlin {
  initial: MAGIC_BARRIER
}
```

5. Sistema de tipos

El lenguaje tendrá un sistema de tipos estricto compuesto por los siguientes tipos fundamentales.

Tipo	Descripción	Ejemplo
Action	Acción de un mago o guerrero	FIREBALL
Integer	Número entero	10
Float	Número decimal	0.25
Boolean	Resultado verdadero o falso	true
List<Action>	Secuencia de acciones	[SLASH, REST]
History	Historial de movimientos	self_history

5.1 Reglas de tipado

Comparaciones numéricas

Los estados siguientes son enteros:


```
self_health
opponent_health
self_resource
opponent_resource
self_score
opponent_score
round_number
```

Pueden compararse con enteros:

```
self_health <= 30
round_number >= 3
self_score > opponent_scoretotal_rounds
```

El estado **random** es flotante y únicamente puede compararse con valores entre 0.0 y 1.0.

```
random < 0.25
random >= 0.80
```

Comparaciones de acciones

Las funciones que devuelven una acción deben compararse con acciones:

```
last_move(opponent_history) == FIREBALL
```

No será válido conceptualmente comparar una acción con un número:

```
last_move(opponent_history) == 10
```

Comparaciones de listas

get_last_n_moves devuelve una lista de acciones.

```
get_last_n_moves(self_history, 3)
== [ARCANE_BOLT, ARCANE_BOLT, FIREBALL]
```

Expresiones booleanas

Toda condición dentro de un if debe devolver un valor booleano.

```
if self_health <= 30 then HEALING_RUNE
```

Restricción por clase

Las acciones colocadas en initial, then y else deben corresponder a la clase de estrategia.

Ejemplo correcto:

```
mage Merlin {
  initial: FIREBALL
}
```

Ejemplo incorrecto:

```
mage Merlin {
  initial: HEAVY_STRIKE
}
```

6. Sección de definición de estrategias

6.1 Acciones fundamentales

Las acciones serán predeterminadas. Los estudiantes no deberán permitir la creación libre de nuevas habilidades.

Esta restricción mantiene el proyecto dentro de una dificultad media-alta y evita la necesidad de construir un analizador semántico complejo.

Estadísticas de las clases

Mago

Estadística	Valor
Vida máxima	100
Maná máximo	120
Ataque físico	5
Poder mágico	25
Armadura	8
Resistencia mágica	18
Velocidad	14

Guerrero

Estadística	Valor
Vida máxima	140
Energía máxima	100
Ataque físico	22
Poder mágico	0
Armadura	20
Resistencia mágica	8
Velocidad	10

Acciones Mago

Acción	Tipo	Poder/efecto	Costo	Prioridad
ARCANE_BOLT	Ataque mágico	12	10 maná	4
FIREBALL	Ataque mágico	25	30 maná	2
MAGIC_BARRIER	Defensa	Reduce 50 %	20 maná	7
HEALING_RUNE	Curación	Recupera 25 HP	30 maná	5
MEDITATE	Recuperación	Recupera 25 maná	0	1

ARCANE_BOLT

Ataque mágico básico. Consume poco maná y tiene prioridad media.

FIREBALL

Ataque mágico de mayor potencia. Consume más maná y tiene menor prioridad.

MAGIC_BARRIER

Reduce en un 50 % el daño recibido durante la ronda actual.

HEALING_RUNE

Recupera hasta 25 puntos de vida. La vida no puede superar el máximo de la clase.

MEDITATE

Recupera hasta 25 puntos de maná.

Acciones Guerrero

Acción	Tipo	Poder/efecto	Costo	Prioridad
SLASH	Ataque físico	12	10 energía	4
HEAVY_STRIKE	Ataque físico	25	25 energía	2
SHIELD_BLOCK	Defensa	Reduce 50 %	15 energía	7
WAR_CRY	Mejora	+10 al próximo ataque	20 energía	6
REST	Recuperación	Recupera 25 energía	0	1

SLASH

Ataque físico básico.

HEAVY_STRIKE

Ataque físico fuerte que consume más energía.

SHIELD_BLOCK

Reduce en un 50 % el daño recibido durante la ronda actual.

WAR_CRY

Aumenta en 10 puntos el ataque físico del siguiente ataque ofensivo ejecutado correctamente. La mejora permanece activa hasta ser consumida.

REST

Recupera hasta 25 puntos de energía.

6.2 Elementos clave

Cada estrategia tendrá los siguientes elementos:

Nombre

Identificador único de la estrategia.

mage Merlin

Clase

Determina las estadísticas y acciones permitidas.

mage
warrior

Acción inicial

Es la acción ejecutada automáticamente durante la ronda número 0.

initial: ARCANE_BOLT

La acción inicial no evalúa reglas.

Reglas

Colección ordenada de condiciones y acciones.

```
rules: [  
  if self_health <= 30 then HEALING_RUNE,  
  if self_resource <= 20 then MEDITATE,  
  else ARCANE_BOLT  
]
```

Acción por defecto

El bloque else deberá aparecer exactamente una vez y siempre al final.

6.3 Sintaxis de estrategia

Estrategia de mago

```
mage <nombre> {  
  initial: <acción de mago>  
  
  rules: [  
    if <condición> then <acción de mago>,  
    if <condición> then <acción de mago>,  
    else <acción de mago>  
  ]  
}
```

Ejemplo

```
mage Merlin {  
  initial: ARCANE_BOLT  
  
  rules: [  
    if self_health <= 30  
      && self_resource >= 30  
    then HEALING_RUNE,  
  
    if opponent_health <= 40  
      && self_resource >= 30  
    then FIREBALL,  
  
    if self_resource <= 20  
    then MEDITATE,  
  
    else ARCANE_BOLT  
  ]  
}
```

Reglas de evaluación

Las reglas se evaluarán en cascada.

Primera regla



¿Es verdadera?

- └ Sí: seleccionar acción y detener evaluación
- └ No: evaluar siguiente regla

Cuando ninguna condición sea verdadera, se utilizará el else.

Las decisiones serán síncronas:

1. Se evalúa la estrategia del jugador 1.
2. Se evalúa la estrategia del jugador 2.
3. Se guardan ambas acciones.
4. Después se ejecutan.

6.4 Operadores

Operadores de comparación

`==, !=, >, <, >=, <=`

Operadores booleanos

`&&, ||, !`

Agrupación

`()`

Precedencia

El orden de precedencia será:

1. Paréntesis y funciones.
2. Negación `!`.
3. Comparaciones.
4. Conjunción `&&`.
5. Disyunción `||`.

Ejemplo:

```
self_resource >= 30
&& (opponent_health <= 35 || random < 0.15)
```

Evaluación de corto circuito

Operador &&

Si la expresión izquierda es falsa, no se evalúa la derecha.

```
round_number >= 3
```

```
&& get_last_n_moves(self_history, 3) == [...]
```

Durante las primeras rondas, la función no se ejecuta porque `round_number >= 3` es falso.

Operador ||

Si la expresión izquierda es verdadera, no se evalúa la derecha. Esto es importante para evitar errores al acceder a historiales insuficientes.

6.5 Funciones del sistema

1. `get_move(history, n)`

Tipo de retorno

Action

Descripción

Obtiene la acción localizada en la posición `n` del historial.

Los índices comienzan en cero.

Sintaxis

0 1 2 3 4

```
get_move(<historial>, <entero>)
```

Ejemplo

```
get_move(opponent_history, 0) == MAGIC_BARRIER
```

Error

El programa deberá terminar si:

- `n` es negativo.
- `n` es mayor o igual al tamaño del historial.

2. `last_move(history)`

Tipo de retorno

Action

Descripción

Obtiene la última acción ejecutada correctamente dentro del historial.

Ejemplo

```
last_move(opponent_history) == FIREBALL
```

Error

Si el historial está vacío, la ejecución deberá terminar.

Por esta razón se recomienda proteger la función:

```
round_number >= 1  
&& last_move(opponent_history) == FIREBALL
```

3. get_moves_count(history, action)

Tipo de retorno

Integer

Descripción

Cuenta cuántas veces aparece una acción dentro de un historial.

Ejemplo

```
get_moves_count(opponent_history, FIREBALL) >= 2
```

También podrá recibir el resultado de otra función:

```
get_moves_count(  
    get_last_n_moves(opponent_history, 3),  
    HEAVY_STRIKE) >= 2
```

4. get_last_n_moves(history, n)

Tipo de retorno

List<Action>

Descripción

Devuelve las últimas n acciones de un historial.

Ejemplo

```
get_last_n_moves(self_history, 3)
== [SLASH, SLASH, HEAVY_STRIKE]
```

Restricciones

- n debe ser mayor que cero.
- n no puede ser mayor que la cantidad de acciones disponibles.

Ejemplo funciones anidadas

```
get_moves_count(
    get_last_n_moves(opponent_history, 3),
    HEAVY_STRIKE
)
```

6.6 Estados del sistema

round_number

- Tipo: Integer.
- Número de la ronda actual.
- Comienza en 0.

round_number >= 3

La ronda 0 utiliza la acción inicial.

total_rounds

- Tipo: Integer.
- Cantidad máxima de rondas de la partida.

round_number == total_rounds - 1

self_health

- Tipo: Integer.
- Vida actual de la estrategia que se está evaluando.

self_health <= 25

opponent_health

- Tipo: Integer.
- Vida actual del rival.

opponent_health <= 30

self_resource

- Tipo: Integer.
- Maná actual si la estrategia es mago.
- Energía actual si es guerrero.

self_resource >= 30

opponent_resource

- Tipo: Integer.
- Recurso actual del oponente.

opponent_resource <= 15

self_score

- Tipo: Integer.
- Puntuación acumulada propia.

self_score < opponent_score

opponent_score

- Tipo: Integer.
 - Puntuación acumulada del rival.
-

self_history

- Tipo: History.
 - Historial de acciones ejecutadas correctamente por el personaje.
-

opponent_history

- Tipo: History.
 - Historial de acciones ejecutadas correctamente por el oponente.
-

random

- Tipo: Float.
- Valor comprendido entre 0.0 y 1.0.
- Se genera una vez por combatiente al inicio de cada ronda posterior a la ronda inicial.

random < 0.20

Todas las referencias a random de un combatiente durante una misma ronda utilizarán el mismo valor.

7. Sección de definición de partidas

Las partidas serán estrictamente uno contra uno.

Se permitirán las siguientes combinaciones:

- Mago contra guerrero.
- Mago contra mago.
- Guerrero contra guerrero.

Cada partida deberá definir:

- Nombre.
- Dos participantes.
- Número máximo de rondas.
- Sistema de puntuación.
- Bonificaciones

Ejemplo:

```
match DueloDeLaTorre {  
  players: [Merlin, Ragnar]  
  
  rounds: 10  
  
  scoring: {  
    damage_point: 1,  
    healing_point: 1,  
    successful_defense: 20,  
    victory_bonus: 100,  
    failed_action_penalty: 10  
  }  
  
  bonuses: {  
    mage_combo: [ARCANE_BOLT, ARCANE_BOLT, FIREBALL],  
    mage_combo_points: 35,  
    warrior_combo: [SLASH, SLASH, HEAVY_STRIKE],  
    warrior_combo_points: 40,  
    low_health_victory: 25  
  }  
}
```

7.2 Reglas de puntuación

Puntos por daño

Por cada punto de daño real causado se otorgará:

daño real $\times$ damage_point

Ejemplo:

Daño real: 29
damage_point: 1
Puntos obtenidos: 29

El daño real es el daño final después de aplicar defensas.

Puntos por curación

Se otorgarán puntos únicamente por la vida recuperada realmente.

vida recuperada $\times$ healing_point

Ejemplo:

Vida actual: 90
Vida máxima: 100
HEALING_RUNE recupera hasta 25
Vida recuperada realmente: 10

Solamente se otorgarán puntos por esos 10 puntos.

Defensa exitosa

Si MAGIC_BARRIER o SHIELD_BLOCK reduce al menos un punto de daño, el personaje obtiene:

successful_defense

Si el rival no realiza un ataque, la defensa no entrega puntos.

Bonificación por victoria

El ganador obtiene:

victory_bonus

Penalización por acción fallida

Cuando un combatiente selecciona una acción y no tiene suficiente recurso:

- La acción no produce efecto.
- No se agrega al historial.
- Se resta failed_action_penalty.
- La puntuación no puede quedar por debajo de cero.

nuevoPuntaje =

$\max(0, \text{puntajeActual} - \text{failed_action_penalty})$

Restricciones de puntuación

- Todos los valores deben ser enteros no negativos.
- damage_point debe ser mayor que cero.
- Cada propiedad deberá aparecer exactamente una vez.
- No se permitirán propiedades adicionales.
- La penalización se define como positiva, pero se resta durante la ejecución.

7.3 Bonificaciones y combos

Cada partida define una secuencia para magos y una para guerreros.

Combo de mago

mage_combo: [ARCANE_BOLT, ARCANE_BOLT, FIREBALL]

Combo de guerrero

warrior_combo: [SLASH, SLASH, HEAVY_STRIKE]

Cuando las últimas acciones ejecutadas correctamente coincidan con la secuencia, se otorgan los puntos correspondientes.

mage_combo_points: 35

warrior_combo_points: 40

Los combos podrán activarse más de una vez durante una partida, pero una misma acción final no podrá entregar el mismo bonus dos veces.

Victoria con poca vida

Se otorgará:

low_health_victory

cuando el ganador termine la partida con una vida igual o inferior al 25 % de su vida máxima.

Mago

25 % de 100 = 25 HP

Guerrero

25 % de 140 = 35 HP

7.4 Mecánica de combate

Fórmula de daño físico

daño =

- poder de la acción
- + ataque físico del atacante
- + bonificación de WAR_CRY
- armadura del defensor

El resultado mínimo será uno:

$\text{dañoFinal} = \max(1, \text{daño})$

Ejemplo

Ragnar utiliza SLASH contra Merlin:

Poder de SLASH: 12

Ataque físico de Ragnar: 22

Armadura de Merlin: 8

$\text{Daño} = 12 + 22 - 8$

$\text{Daño} = 26$

Fórmula de daño mágico

daño =

- poder de la acción
- + poder mágico del atacante
- resistencia mágica del defensor

Ejemplo

Merlin utiliza FIREBALL contra Ragnar:

Poder de FIREBALL: 25

Poder mágico de Merlin: 25

Resistencia mágica de Ragnar: 8

Daño = $25 + 25 - 8$

Daño = 42

Reducción por defensa

Una defensa reduce el daño final en 50 %.

$\text{dañoDefendido} = \text{floor}(\text{dañoOriginal} \times 0.50)$

Ejemplo:

Daño original: 29

Daño defendido: $\text{floor}(29 \times 0.50)$

Daño defendido: 14

Todo cálculo decimal deberá truncarse hacia abajo.

Prioridad de acciones

La acción con mayor prioridad se ejecutará primero.

Acción	Prioridad
MAGIC_BARRIER	7
SHIELD_BLOCK	7
WAR_CRY	6
HEALING_RUNE	5
ARCANE_BOLT	4
SLASH	4
FIREBALL	2
HEAVY_STRIKE	2
MEDITATE	1
REST	1

Cuando ambas acciones tengan la misma prioridad:

1. Actúa primero el personaje con mayor velocidad.
2. Si tienen la misma velocidad, actúa primero el jugador colocado en la primera posición de players.

7.5 Condiciones de victoria

Una partida puede terminar de dos formas.

Derrota directa

La partida termina inmediatamente cuando la vida de un combatiente llega a cero.

Límite de rondas

Si ambos conservan vida después de completar todas las rondas, se aplicará el siguiente orden:

1. Mayor puntuación.
 2. Mayor vida restante.
 3. Mayor recurso restante.
 4. Si todos los valores son iguales, la partida termina en empate.
-

8. Sección de punto de entrada

8.1 Conceptos clave

El bloque main determinará:

- Qué partidas deben ejecutarse.
- El orden de ejecución.
- La semilla utilizada en cada grupo de partidas.

Podrán existir varias instrucciones run.

8.2 Sintaxis del punto de entrada

```
main {  
  run [<partida 1>, <partida 2>] with {  
    seed: <entero>  
  }  
  
  run [<partida 3>] with {  
    seed: <entero>  
  }  
}
```

Ejemplo

```
main {  
    run [DueloDeLaTorre, BatallaDelCastillo] with {  
        seed: 42  
    }  
  
    run [DueloDeLaTorre] with {  
        seed: 99  
    }  
}
```

Orden de ejecución:

1. DueloDeLaTorre con seed 42
2. BatallaDelCastillo con seed 42
3. DueloDeLaTorre con seed 99

8.3 Validación de parámetros

Seed

La semilla deberá:

- Ser un entero.
- Ser mayor que cero.

seed: 42

Ejemplos inválidos:

seed: -5

seed: 0

seed: 3.14

Lista de partidas

La lista deberá contener al menos una partida.

```
run [DueloFinal] with {  
    seed: 42  
}
```

Una partida inexistente deberá producir un error de ejecución y terminar el programa.

Rondas

La cantidad de rondas deberá ser mayor que cero.

8.4 Reproducibilidad mediante seed

Cada partida deberá inicializar dos generadores independientes:

Jugador 1: seed

Jugador 2: seed + 1

Ejemplo:

Seed general: 42

Jugador 1:

seed = 42

Jugador 2:

seed = 43

Al inicio de cada ronda posterior a la ronda inicial:

1. Se obtiene un valor para el jugador 1.
2. Se obtiene un valor para el jugador 2.
3. Los valores se almacenan.
4. Todas las referencias a random de esa ronda utilizan el valor almacenado.

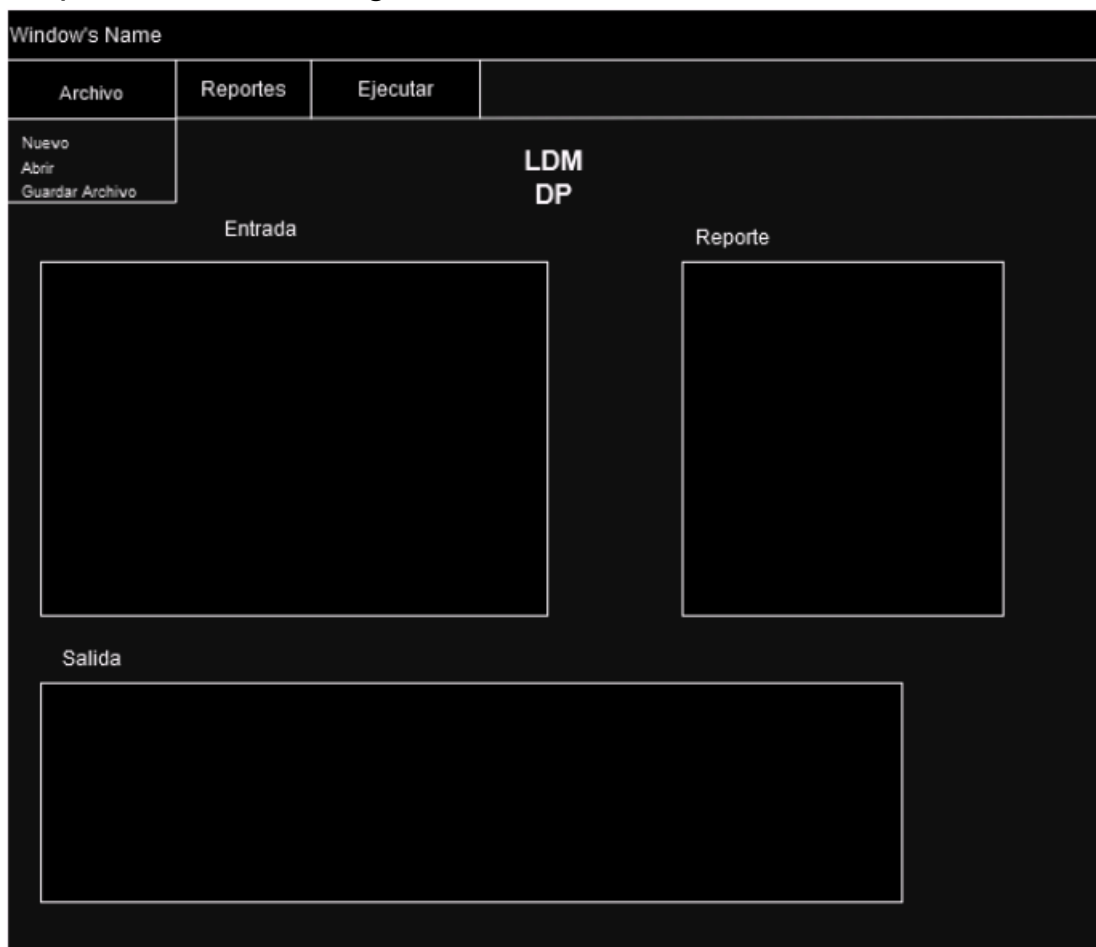
Se generará exactamente un valor por jugador y por ronda, incluso cuando la estrategia no utilice random.

9. Entorno de Trabajo

9.1 Editor

El editor será parte del entorno de trabajo, cuya finalidad será proporcionar ciertas funcionalidades y herramientas que serán de utilidad para el usuario. La función principal del editor será el ingreso del código fuente que será analizado. Queda a discreción del estudiante el diseño.

Propuesta de interfaz grafica



9.2 Funcionalidades

- Nuevo Archivo: se podrá crear un nuevo archivo con extensión.
- Abrir: Abre un explorador de archivos y selecciona uno para seleccionar su información.
- Guardar: nos permitirá guardar el archivo actual.

9.3 Herramientas

- Ejecutar: se envía la entrada actual al intérprete con la finalidad de realizar el análisis léxico, sintáctico y la ejecución de instrucciones.

9.4 Reportes

- Reporte de Tokens: se mostrarán todos los tokens reconocidos en el analizador léxico.
- Reporte de Error: se mostrará el error que es y si es tipo léxico

10. Funcionamiento de la semilla aleatoria

La semilla, identificada mediante la palabra reservada `seed`, es un número entero positivo utilizado para inicializar los generadores de números pseudoaleatorios de una partida.

Su propósito es garantizar la **reproducibilidad de los combates**. Esto significa que, cuando se ejecuta el mismo archivo de entrada con la misma semilla y bajo las mismas reglas de ejecución, el sistema debe producir exactamente los mismos resultados.

Ejemplo:

```
main {  
  run [DueloDeLaTorre] with {  
    seed: 42  
  }  
}
```

El valor 42 no representa una probabilidad, porcentaje, cantidad de rondas ni bonificación. Es únicamente el valor inicial utilizado por el generador pseudoaleatorio.

El proyecto de referencia también utiliza una semilla para garantizar que las ejecuciones puedan repetirse y producir resultados comparables.

11. Reportes

Los reportes son parte fundamental del sistema, ya que muestra de forma visual las herramientas utilizadas para realizar la ejecución del código. Cada reporte debe ser generado luego de cada ejecución por lo que no deberá mostrarse reportes de análisis anteriores. A continuación, se muestran ejemplos de estos reportes. Queda a discreción del estudiante el diseño de estos, solo se pide que sean totalmente legibles.

11.2 Tabla de tokens

El reporte de tokens debe tener la información suficiente para poder identificar los tokens reconocidos en el análisis léxico. Este debe ser en una tabla.

#	Lexema	Tipo	Línea	Columna
1	arr	id	5	3
2	3	Double	6	3
3	"Hola"	String	8	3

11.3 Reporte de error El reporte de error debe contener la información suficiente para detectar y corregir errores en el código fuente.

#	Tipo	Descripción	Línea	Columna
1	Léxico	El carácter "\$" no pertenece al lenguaje	5	3

12. Requerimientos mínimos

Para tener derecho a calificación, el proyecto deberá cumplir con:

- Interfaz gráfica funcional.
- Apertura de archivos .btl.
- Analizador léxico con JFlex.
- Analizador sintáctico con CUP.
- Documentación completa.

+ Generadores con
task

13. Ponderacion

Evaluación	Puntos
Archivo básico	15
Archivo intermedio	25
Archivo avanzado	35
Preguntas sobre implementación	10
Reporte de tokens y errores	8
Manuales	7
Total	100

13. Entregables

El estudiante deberá entregar los siguientes elementos:

- Código fuente completo del proyecto.
- Último hash de Git realizado antes de la fecha límite de entrega.
- Manual de usuario en formato Markdown (.md).
 - Debe incluir capturas de pantalla que expliquen detalladamente el funcionamiento del entorno de trabajo.
 - Debe mostrar el proceso para crear, abrir, guardar y ejecutar archivos.
 - Debe incluir capturas de los reportes generados por la aplicación.
- Manual técnico en formato Markdown (.md).
 - Debe incluir la información necesaria para facilitar el mantenimiento futuro del proyecto.
 - Debe especificar el lenguaje de programación utilizado.
 - Debe indicar las herramientas y librerías empleadas.
 - Debe describir la estructura general del proyecto.
 - Debe explicar las clases, métodos y funciones más importantes.
 - Debe explicar el funcionamiento del analizador léxico, analizador sintáctico y motor de ejecución.
- Archivo de gramática en formato Markdown (.md).
 - Debe contener la gramática completa del lenguaje.

- La gramática debe presentarse de forma limpia, ordenada y comprensible.
 - No debe ser una copia directa del archivo de CUP.
 - La gramática debe estar escrita utilizando el formato BNF (*Backus-Naur Form*).
 - Archivo de configuración de JFlex.
 - Archivo de configuración de CUP.
-

14. Restricciones

- La entrega deberá realizarse mediante la plataforma UEDI, enviando el enlace del repositorio privado de GitHub donde se encuentre alojado el proyecto.
- El repositorio deberá nombrarse utilizando el siguiente formato:

OLC1_Proyecto1_#Carnet

Ejemplo:

OLC1_Proyecto1_202012345

- Se deberá agregar al auxiliar encargado como colaborador del repositorio de GitLab.
- El auxiliar deberá contar con permisos de desarrollador (Developer).

Usuarios de GitLab por sección

- **Sección B — 002Fer**
- **Sección C — JohnAlds**
- **Sección N — LempDnote**

Tecnologías obligatorias

- Lenguaje de programación: **Java**.
- Herramienta para el análisis léxico: **JFlex**.
- Herramienta para el análisis sintáctico: **CUP**.

15. Fecha de entrega

La fecha límite de entrega será:

Lunes, 31 de agosto de 2026, a las 23:59 horas.

La entrega deberá realizarse mediante la plataforma **UEDI**, enviando el enlace del repositorio privado de GitHub.

Penalizaciones

- 50% por entrega tarde en UEDI
- 50% al no comprobar autoría
- 15% al no enviar invitación en github

No se calificarán entregas realizadas después de la fecha y hora establecidas.